OOP — Interview Questions & Answers

Q1. What is OOP?

OOP (Object-Oriented Programming) is a programming approach where we organize a program around objects and classes.

An object contains:

- Data → properties/state
- Methods → behavior/actions

OOP helps us structure large programs so they are easier to develop, maintain, and reuse.

Example:

A Car can be an object.

```
class Car {
   String color;
   void drive() {
      System.out.println("Car is driving");
   }
}
```

Here:

color → data
drive() → behavior

Q2. Why do we use OOP?

We use OOP to make programs modular, reusable, maintainable, and easier to manage, especially when applications become large.

It allows us to model real-world entities as objects and provides features such as encapsulation, inheritance, and polymorphism.

Example:

In a banking application, we can have: Customer, Account, Transaction, Loan.

Each can have its own data and behavior instead of putting everything into one huge program.

🔹 Four Pillars of OOP

Q3. What are the 4 pillars of OOP?

The four main principles are:

1. Encapsulation → Bundling data and methods and controlling access to the data.
2. Abstraction → Hiding unnecessary implementation details.
3. Inheritance → Creating a new class based on an existing class.
4. Polymorphism → One interface/name can represent different behaviors.

Example:

Think about a car:

Encapsulation → Car controls its internal data
Abstraction → You use steering/brakes without knowing internal implementation
Inheritance → ElectricCar can inherit from Car
Polymorphism → different cars can implement drive() differently

Q4. What is encapsulation?

Encapsulation means bundling data and the methods that operate on that data together, while controlling direct access to the data.

Usually, we achieve this using private fields and public methods such as getters/setters.

Example:

```
class BankAccount {
  private double balance;
  public void deposit(double amount) {
    if (amount > 0) {
      balance += amount;
    }
  }
  public double getBalance() {
    return balance;
  }
}
```

Here, outside code cannot directly do:

account.balance = -5000;

because balance is private.

Instead, it must go through deposit().

Real-world example:
An ATM doesn't allow you to directly change the bank's database balance. You perform controlled operations such as deposit or withdrawal.

Q5. What is abstraction?

Abstraction means hiding unnecessary implementation details and exposing only what is necessary to the user.

The user knows what to do, without needing to know how it is internally implemented.

Example:

```
interface Payment {
  void pay();
}
```

A user can call:

payment.pay();

without knowing the internal details of how the payment is processed.

Real-world example:
When you use an ATM, you select "Withdraw ₹5,000". You don't need to know how the bank internally verifies your account and processes the transaction.

Q6. What is inheritance?

Inheritance allows one class to acquire properties and behavior from another class.

It promotes code reuse and represents an "is-a" relationship.

Example:

```
class Animal {
  void eat() {
    System.out.println("Eating");
  }
}

class Dog extends Animal {
  void bark() {
    System.out.println("Barking");
  }
}
```

Dog inherits eat() from Animal.

So:

Dog → eat()
Dog → bark()

Real-world example:
A Dog is an Animal, so it can inherit common animal behavior.

Q7. What is polymorphism?

Polymorphism means "many forms." It allows the same method/interface to represent different behaviors depending on the situation.

The two common forms in Java are:

- **Compile-time polymorphism → method overloading**
- **Runtime polymorphism → method overriding**

Example:

```
class Animal {
  void sound() {
    System.out.println("Animal sound");
  }
}
class Dog extends Animal {
```

```
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

If we call:

```
Animal a = new Dog();
a.sound();
```

the output is:

Dog barks

The same sound() method behaves differently depending on the actual object.

🧩 Compile-Time vs Runtime Polymorphism

Q8. What is compile-time vs runtime polymorphism?

Compile-time polymorphism

The method to execute is determined during compilation.

It is commonly achieved through method overloading.

Runtime polymorphism

The method to execute is determined during runtime based on the actual object.

It is achieved through method overriding.

Example — Compile-time:

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

Same method name:

```
add(int, int)
add(int, int, int)
```

The compiler determines which one matches.

Runtime example:

```
Animal a = new Dog();
a.sound();
```

The JVM calls Dog's implementation at runtime.

🔁 Overloading vs Overriding

Q9. What is the difference between method overloading and method overriding?

Method Overloading

Same method name but different parameter list, usually within the same class.

```
void print(int x) { }
void print(String x) { }
```

Method Overriding

A subclass provides its own implementation of a method inherited from its parent class.

```
class Animal {
  void sound() {
    System.out.println("Animal sound");
  }
}
class Dog extends Animal {
  @Override
  void sound() {
    System.out.println("Bark");
  }
}
```

Easy way to remember:

Overloading → same class, different parameters
Overriding → child class changes parent behavior

🎯 Dynamic Binding

Q10. What is dynamic binding?

Dynamic binding means that the method implementation to execute is determined at runtime rather than compile time.

It is associated with method overriding and runtime polymorphism.

Example:

```
Animal animal = new Dog();
animal.sound();
```

The reference type is:

Animal

but the actual object is:

Dog

At runtime, Java chooses Dog's sound() method.

Real-world example:
You say "make a sound" to an animal. If the actual animal is a dog, it barks; if it is a cat, it meows.

🔌 Interface

Q11. What is an interface?

An interface defines a contract that implementing classes agree to follow.

It is commonly used to achieve abstraction and support flexible designs.

Example:

```
interface Payment {
   void pay();
}
class CreditCardPayment implements Payment {
   public void pay() {
      System.out.println("Paid using credit card");
   }
}
```

Any class implementing Payment must provide the required behavior for pay().

Real-world example:
A payment system might define:

Payment
↓
Credit Card
UPI
PayPal

All provide a pay() operation, but each can implement it differently.

🆚 Abstract Class vs Interface

Q12. What is the difference between an abstract class and an interface?

An abstract class is a partially implemented class that can contain state, constructors, concrete methods, and abstract methods.

An interface primarily defines a contract. In modern Java, interfaces can also contain default, static, and private methods with implementations.

- **Abstract Class:** Can have instance variables/state, constructors, abstract + concrete methods, and extends only one class.
- **Interface:** Fields are constants, no constructors, supports multiple implementation, and defines a contract.

Example:

```
abstract class Animal {
  String name;
  Animal(String name) {
    this.name = name;
  }
  abstract void sound();
  void sleep() {
    System.out.println("Sleeping");
  }
}
```

Interface:

```
interface Flyable {
  void fly();
}
```

A class could do:

```
class Bird extends Animal implements Flyable {
  // ...
}
```

So it can inherit from one class while implementing multiple interfaces.

🔗 Composition vs Inheritance

Q13. What is composition vs inheritance?

Inheritance represents an "is-a" relationship.

Composition represents a "has-a" relationship, where one object contains or uses another object.

Inheritance:

```
class Animal { }

class Dog extends Animal { }
```

A dog is an animal.

Composition:

```
class Engine { }

class Car {
  private Engine engine = new Engine();
}
```

A car has an engine.

Important interview point:

> Prefer composition over inheritance when you don't have a strong "is-a" relationship, because composition generally gives more flexibility and reduces tight coupling.

🤝 Association

Q14. What is association?

Association is a general relationship between two independent objects.

One object knows about or interacts with another object.

Example:

```
class Teacher {
}
class Student {
  Teacher teacher;
}
```

A student has a relationship with a teacher.

Neither object necessarily owns the lifetime of the other.

Real-world example:
A doctor and patient have an association. They interact, but a doctor doesn't own the patient's existence.

🪢 Aggregation

Q15. What is aggregation?

Aggregation is a weak "has-a" relationship where the contained object can exist independently of the container.

Example:

```
class Student {
}
class Department {
  List<Student> students;
```

}

A department has students, but students can exist even if the department object is removed.

Real-world example:
A college and students can be considered an aggregation: students can continue to exist even if a particular college object is no longer represented in the program.

🧱 Composition

Q16. What is composition?

Composition is a strong "has-a" relationship where the contained object's lifecycle is closely tied to the containing object.

Example:

```
class Engine {
}
class Car {
   private Engine engine = new Engine();
}
```

The Car creates and owns its Engine in this example.

Real-world example:
Think of a house and its rooms in a simplified model. The rooms are considered parts of the house.

Easy comparison:

Association → knows/uses
Aggregation → has-a, weak ownership
Composition → has-a, strong ownership

🏗️ Constructors

Q17. What are constructors?

A constructor is a special member of a class that is called when an object is created. It is mainly used to initialize the object's state.

In Java:

- **Constructor has the same name as the class.**
- **It has no return type.**
- **It runs when an object is created.**

Example:

```
class Student {
   String name;
```

```
  Student(String name) {
    this.name = name;
  }
}
Student s = new Student("Rahul");
```

When the object is created, the constructor initializes name.

Q18. Can constructors be inherited?

No, constructors are not inherited by subclasses.

However, when a child object is created, the parent constructor is invoked as part of the initialization process, either explicitly or implicitly through super().

Example:

```
class Animal {
  Animal() {
    System.out.println("Animal constructor");
  }
}
class Dog extends Animal {
  Dog() {
    System.out.println("Dog constructor");
  }
}
```

When:

```
new Dog();
```

the output is:

```
Animal constructor
Dog constructor
```

The Dog object initialization first invokes the parent constructor.

👉 this

Q19. What is this?

this is a reference to the current object.

It is commonly used when instance variables and method/constructor parameters have the same name.

Example:

```
class Student {
  String name;
```

```
  Student(String name) {
    this.name = name;
  }
}
```

Here:

this.name → object's instance variable
name → constructor parameter

Real-world example:
If I say "my name," my refers to the current person. Similarly, this refers to the current object.

👆 super

Q20. What is super?

super refers to the immediate parent class.

It can be used to:

- **Access a parent class variable.**
- **Call a parent class method.**
- **Call a parent class constructor.**

Example:

```
class Animal {
  String name = "Animal";
}
class Dog extends Animal {
  String name = "Dog";

  void printNames() {
    System.out.println(name);
    System.out.println(super.name);
  }
}
```

Output:

Dog
Animal

Here:

name → Dog's variable
super.name → Animal's variable

🔐 Access Modifiers

Q21. What are access modifiers?

Access modifiers control where classes, methods, and variables can be accessed from.

In Java, the main ones are:

public
protected
default (package-private)
private

private

Accessible only within the same class.

default

Accessible within the same package.

protected

Accessible within the same package and in subclasses, including subclasses in other packages with Java's specific access rules.

public

Accessible from anywhere subject to normal Java visibility rules.

Example:

```
class Account {
   private double balance;
   public double getBalance() {
      return balance;
   }
}
```

We keep balance private and expose controlled access through a public method.

⚡ Static

Q22. What is static?

static means the member belongs to the class rather than to individual objects.

A static member is shared among instances of that class.

Example:

```
class Student {
```

```
    static String college = "ABC College";
    String name;
}
```

All Student objects share the same college variable.

```
Student s1 = new Student();
Student s2 = new Student();
```

Both refer to the same static college.

Real-world example:
If 1,000 students belong to the same college, you don't need to store a separate copy of the college name conceptually for every student. It can be class-level information.

🔒 Final

Q23. What is final?

final is used to restrict modification.

It can be used with variables, methods, and classes.

Final variable

Cannot be reassigned after initialization.

```
final int MAX = 100;
```

Final method

Cannot be overridden by a subclass.

```
final void display() {
}
```

Final class

Cannot be extended.

```
final class Calculator {
}
```

Example:

```
final double PI = 3.14159;
```

You shouldn't reassign PI.

Easy way to remember:

```
final variable → cannot reassign
final method   → cannot override
final class    → cannot extend
```

⭐ SOLID Principles

Q24. Why should we follow SOLID principles?

SOLID principles are guidelines for designing object-oriented software that is easier to maintain, extend, test, and modify.

SOLID stands for:

S → Single Responsibility Principle
O → Open/Closed Principle
L → Liskov Substitution Principle
I → Interface Segregation Principle
D → Dependency Inversion Principle

You don't need to blindly apply them everywhere. They are design guidelines that help reduce tight coupling and unnecessary complexity.

S — Single Responsibility

A class should have one primary responsibility.

Example:
Don't make one Employee class responsible for employee data, generating reports, sending emails, and saving to the database.

O — Open/Closed

Software entities should be open for extension but closed for modification.

Example:
You should be able to add a new payment method without constantly modifying existing payment logic.

L — Liskov Substitution

A subclass should be usable wherever its parent type is expected without breaking the correctness of the program.

Example:
If Dog extends Animal, code expecting an Animal should not suddenly fail because it receives a Dog.

I — Interface Segregation

Don't force a class to implement methods it doesn't need.

Bad:

Animal
→ eat()

→ walk()
→ fly()
→ swim()

A dog shouldn't be forced to implement fly().

It's better to have smaller, focused interfaces.

D — Dependency Inversion

High-level code should depend on abstractions rather than concrete implementations.

Example:

Instead of:

PaymentService service = new CreditCardPayment();

designing around:

Payment payment;

allows the implementation to be changed more easily.

⭐ Q25. "Explain OOP using a real-world example."

This is one you should practice until you can answer naturally.

Interview answer:

> "OOP is a programming approach where we organize a program using classes and objects. For example, in a banking application, we can have a BankAccount class with properties such as account number and balance, and methods such as deposit and withdraw.
>
> We can use encapsulation by keeping the balance private and allowing it to be modified only through methods like deposit and withdraw. We can use abstraction by exposing only the operations the user needs without showing the internal implementation. We can use inheritance if we have different types of accounts, such as SavingsAccount and CurrentAccount. Finally, polymorphism allows different account types to implement the same operation, such as calculating interest, differently.
>
> So OOP helps us organize the application into reusable and maintainable components."

That's a much stronger interview answer than simply saying:

> "OOP has four pillars: encapsulation, abstraction, inheritance and polymorphism."

🎯 OOP Questions You Should Be VERY Strong At

For placements, prioritize these:

1. What is OOP?
2. Why do we use OOP?
3. Four pillars of OOP
4. Encapsulation
5. Abstraction
6. Inheritance
7. Polymorphism
8. Overloading vs Overriding
9. Compile-time vs Runtime Polymorphism
10. Dynamic Binding
11. Interface
12. Abstract Class vs Interface
13. Composition vs Inheritance
14. Association vs Aggregation vs Composition
15. Constructors
16. Can constructors be inherited?
17. this vs super
18. Access Modifiers
19. static
20. final
21. SOLID principles

🔥 And expect these follow-ups

If you say "Inheritance promotes code reuse", the interviewer may ask:

> **"Then why not use inheritance everywhere?"**

If you say "Interface provides abstraction", they may ask:

> **"Can an interface have implemented methods in Java?"**

If you say "Overloading is compile-time polymorphism", they may ask:

> **"Can you overload a method by changing only its return type?"**

If you say "private provides encapsulation", they may ask:

> **"Why can't we just make everything public?"**

So for interviews, understanding why something works is more valuable than memorizing its definition.